

第3章 动态规划

提纲

CONTENTS

1 动态规划概述

2 一维动态规划

3 二维动态规划

4 三维动态规划

5 字符串动态规划

6 背包动态规划

7 树形动态规划

8 区间动态规划

背包动态规划

说明

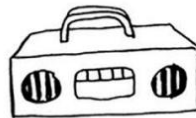
背包动态规划主要指采用动态规划求解0/1背包问题、完全背包问题和多重背包问题及其类似的问题。

假设你背着一个可装

4磅东西的背包。

你可选择的商品有如下3件。

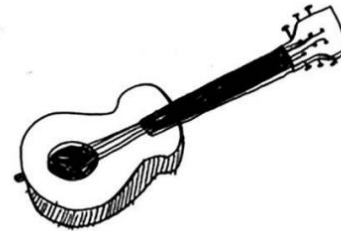
为了让背包里的商品价值最高，你该选择哪些商品？



音响
3000美元
4磅

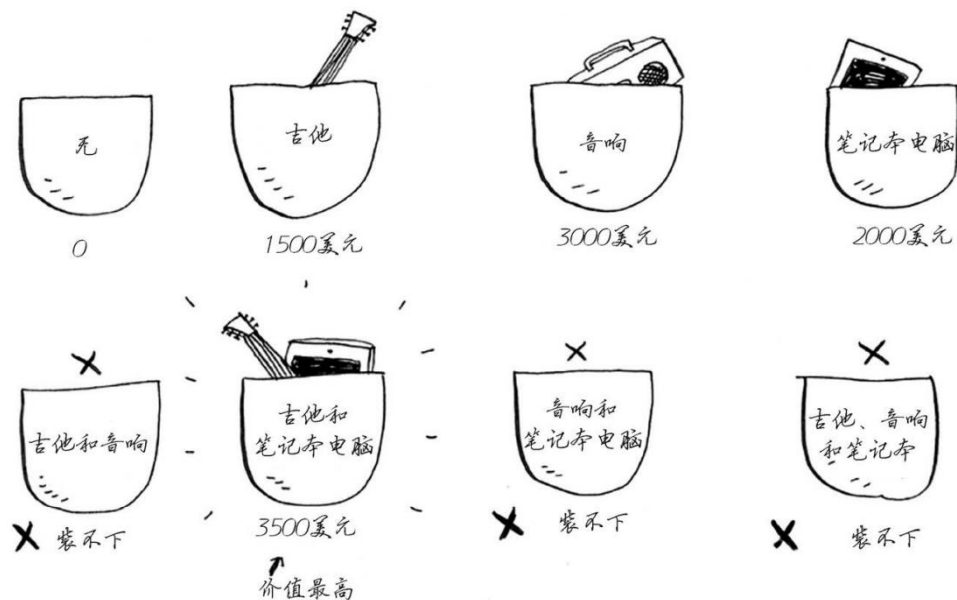


笔记本电脑
2000美元
3磅



吉他
1500美元
1磅

最简单的算法如下：尝试各种可能的商品组合，并找出价值最高的组合。



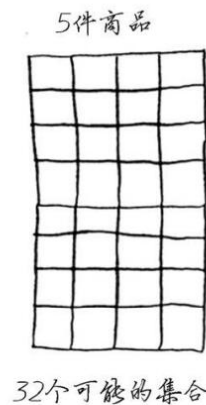
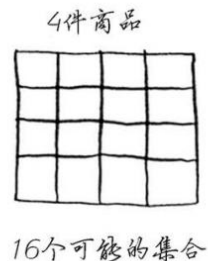
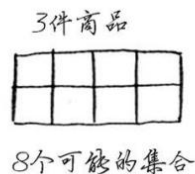
虽可行，但速度非常慢。

有3件商品时，你需要计算8个不同的集合；

有4件商品时，你需要计算16个集合。

每增加一件商品，需要计算的集合数都将翻倍！

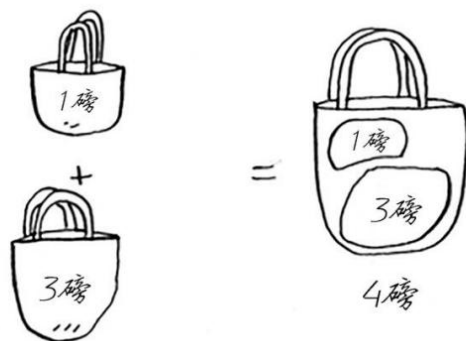
这种算法的运行时间为 $O(2^n)$ ，真的是慢如蜗牛。



如果有32件商品
将会有40亿个
可能的集合！

动态规划先解决子问题，再逐步解决大问题。

对于背包问题，你先解决小背包(子背包) 问题，再逐步解决原来的问题。



每个动态规划算法都从一个网格开始，背包问题的网格如下。

网格的各行为商品，
各列为不同容量(1~4磅) 的背包。
所有这些列你都需要，因为它们
将帮助你计算子背包的价值。
网格最初是空的。
你将填充其中的每个单元格

各列为不同容量(1~4磅) 的背包

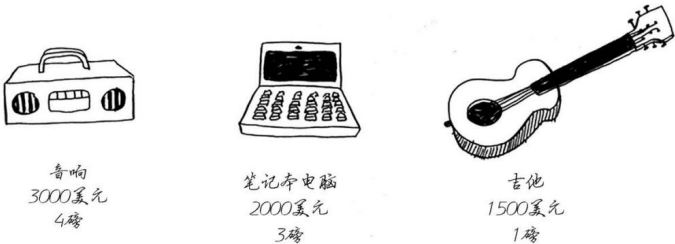
	1	2	3	4
吉他				
音响				
笔记本电脑				

各行为可选择的商品

网格最初是空的。

你将填充其中的每个单元格，网格填满后，就找到了问题的答案！

请你创建网格，我们一起来填满它。



1. 吉他行

首先来看第一行。

对于吉他行：尝试将吉他装入背包。
在每个单元格，都需要做一个简单的决定：
选不选吉他？别忘了，你要找出一个价值最高的商品集合。

	1	2	3	4
吉他				
音响				
笔记本电脑				

	1	2	3	4
吉他 (G)	\$1500 G			
音响				
笔记本电脑				

	1	2	3	4
吉他 (G)	\$1500 G	\$1500 G		
音响				
笔记本电脑				

	1	2	3	4
吉他 (G)	\$1500 G	\$1500 G	\$1500 G	\$1500 G
音响				
笔记本电脑				

当前，为了让背包中商品的价值最高，应选择价值1500美元的吉他

这不是最终的解。
随着算法往下执行，你将逐步修改最大价值。

2. 音响行



音响
3000美元
4磅



笔记本电脑
2000美元
3磅



吉他
1500美元
1磅

在第二行，可选的商品有吉他和音响。

在每一行，可选的商品都为当前行的商品以及之前各行的商品。因此，当前你还不能选笔记本电脑，而只能选音响和吉他。

我们先来看第一个单元格，它表示容量为1磅的背包。在此之前，可装入1磅背包的商品的最大价值为1500美元。

背包容量为1磅时，之前可装入的商品的最大价值

	1	2	3	4
吉他 (G)	\$1500 G	\$1500 G	\$1500 G	\$1500 G
音响				
笔记本电脑				

背包容量为1磅时，现在可装入的商品的最大价值

该不该选音响呢？

背包的容量为1磅，能装下音响吗？

音响太重了，装不下！由于容量1磅的背包装不下音响，

因此最大价值依然是1500美元。

	1	2	3	4
吉他 (G)	\$1500 G	\$1500 G	\$1500 G	\$1500 G
音响	\$1500 G			
笔记本电脑				

如果背包的容量为4磅，就能装入价值至少3000美元的商品。

在这个网格中，逐步地更新最大价值。

	1	2	3	4
吉他 (G)	\$1500 G	\$1500 G	\$1500 G	\$1500 G
音响 (S)	\$1500 G	\$1500 G	\$1500 G	\$3000 S, G
笔记本电脑				

← 以前的最大价值
← 最新的最大价值
← 最终解

3. 笔记本电脑行



音响
3000美元
4磅



笔记本电脑
2000美元
3磅



吉他
1500美元
1磅

笔记本电脑重3磅，没法将其装入容量为1磅或2磅的背包，因此前两个单元格的~~最大~~价值还是1500美元。

	1	2	3	4
吉他 (G)	\$1500 G	\$1500 G	\$1500 G	\$1500 G
音响 (S)	\$1500 G	\$1500 G	\$1500 G	\$3000 S
笔记本电脑	\$1500 G	\$1500 G		

对于容量为3磅的背包，原来的最大价值为1500美元，但现在你可选择价值2000美元的笔记本电脑而不是吉他，这样新的最大价值将为2000美元！

	1	2	3	4
吉他 (G)	\$1500 G	\$1500 G	\$1500 G	\$1500 G
音响 (S)	\$1500 G	\$1500 G	\$1500 G	\$3000 S
笔记本电脑 (L)	\$1500 G	\$1500 G	\$2000 L	

3. 笔记本电脑行



音响
3000美元
4磅



笔记本电脑
2000美元
3磅



吉他
1500美元
1磅

对于容量为4磅的背包，当前的最大价值为3000美元，

你可不选音响，而选笔记本电脑，但它只值2000美元。价值没有原来高。

\$3000 vs \$2000

音响

笔记本电脑

吉他 (G)

音响 (S)

笔记本电脑 (L)

	1	2	3	4
吉他 (G)	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G
音响 (S)	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$3000 ↓ S
笔记本电脑 (L)	\$1500 ↓ G	\$1500 ↓ G	\$2000 ↓ L	

但等一等，笔记本电脑的重量只有3磅，背包还有1磅的容量没用！

\$3000 vs (\$2000 + ???)

音响 笔记本电脑 余下的1磅容量

在1磅的容量中，可装入的商品的最大价值是多少呢？你之前计算过。

1磅容量可
装入商品的
最大价值



	1	2	3	4
吉他 (G)	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G
音响 (S)	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$3000 ↓ S
笔记本电脑 (L)	\$1500 ↓ G	\$1500 ↓ G	\$2000 ↓ L	

\$3000 vs (\$2000 + \$1500)

音响 笔记本电脑 吉他

	1	2	3	4
吉他 (G)	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G
音响 (S)	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$3000 ↓ S
笔记本电脑 (L)	\$1500 ↓ G	\$1500 ↓ G	\$2000 ↓ L	\$3500 ↓ LG

↑
最终答案

为何计算小背包可装入的商品的最高价值呢？

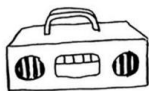
余下了空间时，你可根据这些子问题的答案来确定余下的空间可装入哪些商品。

	1	2	3	4
吉他 (G)	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$1500 G
音响 (S)	\$1500 ↓ G	\$1500 ↓ G	\$1500 G	\$3000 S
笔记本电脑 (L)	\$1500 G	\$1500 G	\$2000 L	\$3500 L G

↑
最终答案

$$\begin{array}{c} \text{行} \\ \downarrow \\ \text{CELL}[i][j] \end{array} \quad \begin{array}{c} \text{列} \\ \downarrow \end{array} = \begin{array}{c} \text{两者中较} \\ \text{大的那个} \end{array} \left\{ \begin{array}{l} 1. \text{上一个单元格的值 (即 CELL}[i-1][j] \text{ 的值)} \\ \text{VS} \\ 2. \text{当前商品的价值} + \text{剩余空间的价值} \end{array} \right.$$

↑
 $\text{CELL}[i-1][j - \text{当前商品的重量}]$



音响
3000美元
4磅



笔记本电脑
2000美元
3磅



吉他
1500美元
1磅



IPHONE
2000美元
1磅

	1	2	3	4
吉他 (G)	\$1500 G	\$1500 G	\$1500 G	\$1500 G
音响 (S)	\$1500 G	\$1500 G	\$1500 G	\$3000 S
笔记本电脑 (L)	\$1500 G	\$1500 G	\$2000 L	\$3500 LG
iPhone				

↑
新的答案

\$1500 G	\$1500 G	\$1500 G	\$1500 G
\$1500 G	\$1500 G	\$1500 G	\$3000 S
\$1500 G	\$1500 G	\$2000 L	\$3500 LG
\$2000 I	\$3500 IG	\$3500 IG	\$4000 IL

↑
新答案

1 0/1背包问题

- 有 n 个编号为 $0 \sim n-1$ 的物品，重量为 $w=\{w_0, w_1, \dots, w_{n-1}\}$ ，价值为 $v=\{v_0, v_1, \dots, v_{n-1}\}$ ，给定一个容量为 W 的背包。
- 从这些物品中选取全部或者部分物品装入该背包中，每个物品要么选中要么不选中，即物品不能被分割，找到选中物品不仅能够放到背包中而且**价值最大**的方案。

物品编号	重量 w	价值 v
0	2	6
1	2	3
2	6	5
3	5	4
4	4	6

$W=10$



求解结果(最优方案)
选取的物品: 0 1 4
总价值=15

解

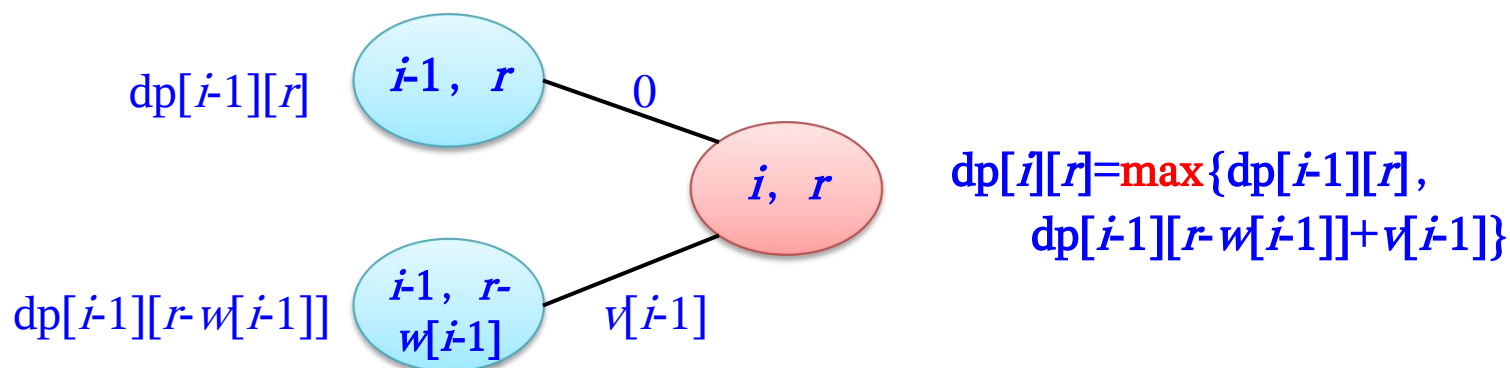
设置二维动态规划数组 dp ， $dp[i][r]$ 表示在物品 $0 \sim i-1$ （共 i 个物品）中选择物品并且背包容量为 r （ $0 \leq r \leq W$ ）时最大价值，或者说只考虑前 i 个物品并且背包容量为 r 时最大价值。

考虑物品 $i-1$ ，分为两种情况：

- 若 $r < w[i-1]$ ，说明物品 $i-1$ 放不下，此时等效于只考虑前 $i-1$ 个物品并且背包容量为 r 时的最大价值，所以有 $dp[i][r] = dp[i-1][r]$ 。

- 若 $r \geq w[i-1]$ ，说明物品 $i-1$ 能够放入背包。有两种选择：
- 不选择物品 $i-1$ 即不将物品 $i-1$ 放入背包，等同于情况①。
 - 选择物品 $i-1$ 即将物品 $i-1$ 放入背包，这样消耗了 $w[i-1]$ 的背包容量，获取了 $v[i-1]$ 的价值，那么留给前 $i-1$ 个物品的背包容量就只有 $r - w[i-1]$ 了，此时的最大价值为 $dp[i-1][r - w[i-1]] + v[i-1]$ 。

两种选择中取最大值。



状态转移方程

$dp[i][0]=0$ （没有装入任何物品，总价值为0）

边界情况

$dp[0][r]=0$ （没有考虑任何物品，总价值为0）

边界情况

$dp[i][r]=dp[i-1][r]$

当 $r < w[i-1]$ 时，物品 $i-1$ 放不下

$dp[i][r] = \max\{dp[i-1][r], dp[i-1][r-w[i-1]]+v[i-1]\}$

否则在不放入和放入物品 $i-1$ 之间取最大价值

在求出 dp 数组后 $dp[n][W]$ 元素就是答案。

```

int n=5,W=10;                                //5种物品,限制重量不超过10
int w[MAXN]={2,2,6,5,4}; int v[MAXN]={6,3,5,4,6};
int dp[MAXN][MAXW];                          //二维动态规划数组
int x[MAXN];
int Knap()                                    //动态规划法求0/1背包问题
{
    for (int i=0;i<=n;i++)                    //边界情况dp[i][0]=0
        dp[i][0]=0;
    for (int r=0;r<=W;r++)                    //边界情况dp[0][r]=0
        dp[0][r]=0;
    for (int i=1;i<=n;i++)
    {
        for (int r=0;r<=W;r++)
        {
            if (r<w[i-1])
                dp[i][r]=dp[i-1][r];
            else
                dp[i][r]=max(dp[i-1][r],dp[i-1][r-w[i-1]]+v[i-1]);
        }
    }
    return dp[n][W];
}

```

【算法分析】 时间复杂度和空间复杂度均为 $O(nW)$ 。是多项式级？

当求出dp数组后，如何推导出一个解向量 $x = (x_0, x_1, \dots, x_{n-1})$ 呢？
从前面状态转移方程的后两行看出：

- 若 $dp[i][r] = dp[i-1][r]$ ，表示物品 $i-1$ 放不下或者不放入物品 $i-1$ 的情况，总之不选择物品 $i-1$ 。也就是说当前dp元素等于上方元素，不选择对应的物品（物品 $i-1$ ），并跳到上方位置。
- 否则一定有 $dp[i][r] \neq dp[i-1][r]$ 成立，表示选择物品 $i-1$ 。也就是说当前dp元素不等于上方元素，选择对应的物品，并跳到左上角 $dp[i-1][r-w[i-1]]$ 的位置。

$$dp[i][r] = dp[i-1][r]$$

当 $r < w[i-1]$ 时，物品 $i-1$ 放不下

$$dp[i][r] = \max\{dp[i-1][r], dp[i-1][r-w[i-1]] + v[i-1]\}$$

否则在不放入和放入物品 $i-1$ 之间取最大价值

$i \backslash r$	0	1	2	3	4	5	6	7	8	9	10
$w_0=2$	0	0	0	0	0	0	0	0	0	0	0
$w_1=2$	1	0	0	6	6	6	6	6	6	6	6
$w_2=6$	2	0	0	6	6	9	9	9	9	9	9
$w_3=5$	3	0	0	6	6	9	9	9	9	11	14
$w_4=4$	4	0	0	6	6	9	9	9	10	11	13
5	0	0	6	6	9	9	12	12	15	15	15

选择物品4
选择物品1
选择物品0

- 若 $dp[i][r] = dp[i-1][r]$ ，当前 dp 元素等于上方元素，不选择物品 $i-1$ ，并跳到上方位置，即 $dp[i][r] \Rightarrow dp[i-1][r]$ 。
- 否则当前 dp 元素等于上方元素，选择对应的物品 $i-1$ ，并跳到左上角 $dp[i-1][r-w[i-1]]$ 的位置，即 $dp[i][r] \Rightarrow dp[i-1][r-w[i-1]]$ 。

```
void getx()
```

//回推求一个最优方案

```
{  int i=n,r=W;
```

```
  while (i>=1)
```

```
  {  if (dp[i][r]!=dp[i-1][r])
```

```
    {  x[i-1]=1;
```

//选取物品i-1

```
      r=r-w[i-1];
```

```
    }
```

```
    else x[i-1]=0;
```

//不选取物品i-1

```
    i--;
```

```
  }
```

```
}
```

程序 验证

```
...  
int main()  
{   int maxv=Knap();  
    printf("dp:\n");  
    for(int i=0;i<=n;i++)  
    {   for(int r=0;r<=W;r++)  
        printf("%3d",dp[i][r]);  
        printf("\n");  
    }  
    getx();  
    printf("求解结果(最优方案)\n");  
    printf("   选取的物品:");  
    for (int i=0;i<n;i++)  
        if (x[i]==1)  
            printf("%d ",i);  
    printf("\n");  
    printf("   总价值=%d\n",maxv);  
}
```



E:\算法基础\源代码\ch8\Kna...

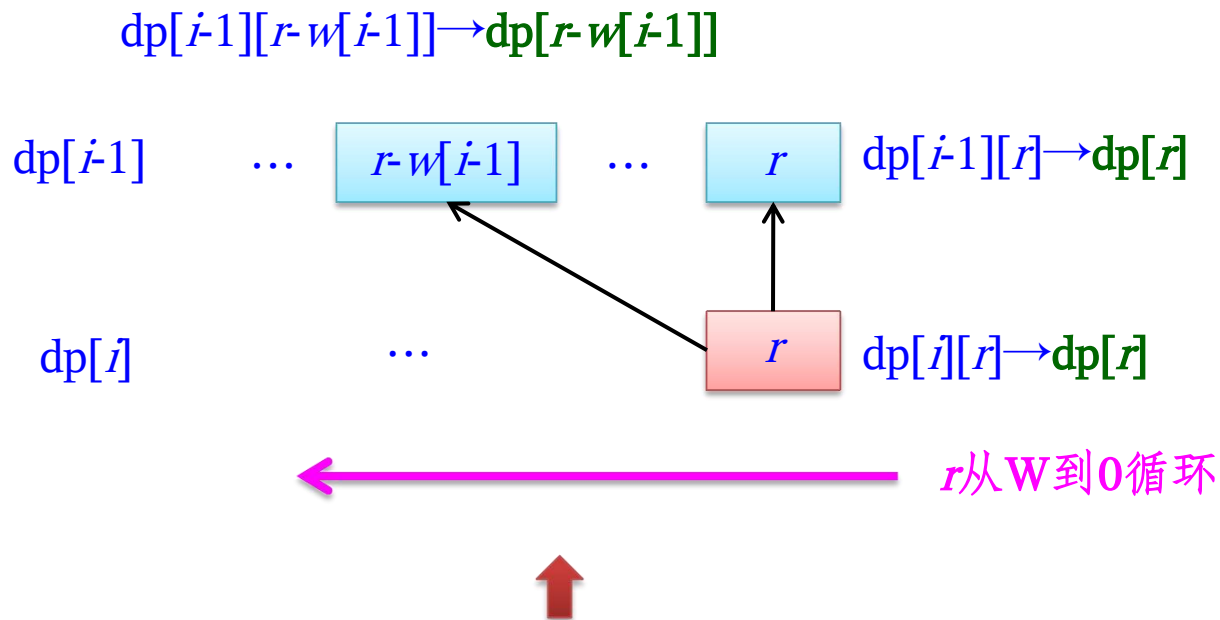
dp:

0	0	0	0	0	0	0	0	0	0
0	0	6	6	6	6	6	6	6	6
0	0	6	6	9	9	9	9	9	9
0	0	6	6	9	9	9	9	11	11
0	0	6	6	9	9	9	10	11	13
0	0	6	6	9	9	12	12	15	15

求解结果<最优方案>
选取的物品: 0 1 4
总价值=15

算法空间优化*

- $dp[MAXN][MAXW]$ 改为一维数组 $dp[MAXW]$ 。



$$dp[i][r] = dp[i-1][r]$$

$$dp[i][r] = \max\{dp[i-1][r], dp[i-1][r-w[i-1]] + v[i-1]\}$$

```
int Knap1()
```

```
{  int dp[MAXN];
```

```
  memset(dp,0,sizeof(dp));
```

```
  for (int i=1;i<=n;i++)
```

```
  {  for (int r=W;r>=0;r--)
```

?

```
    {  if (r<w[i-1])
```

```
        dp[r]=dp[r];
```

```
    else
```

```
        dp[r]=max(dp[r],dp[r-w[i-1]]+v[i-1]);
```

```
    }
```

```
}
```

```
return dp[W];
```

```
}
```

//改进算法

//一维动态规划数组

//置边界情况

//r按1到W的逆序（重点）

可以等价地改为如下

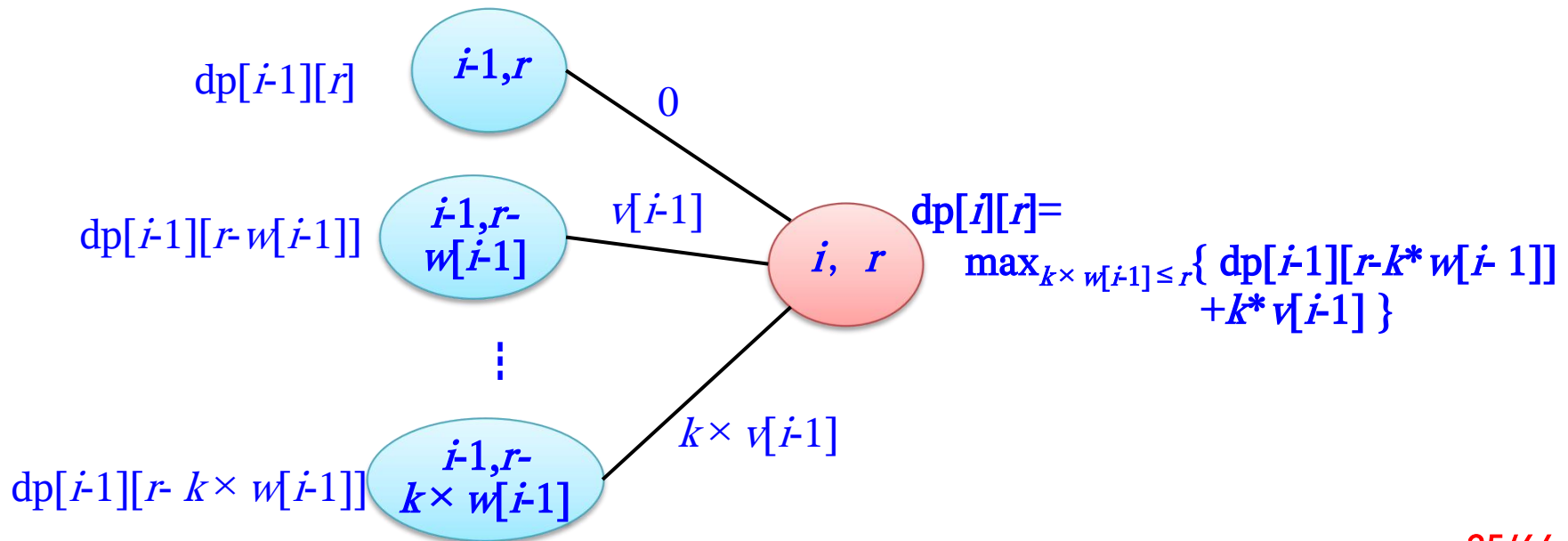
```
int Knap2()                                //改进算法
{  int dp[MAXN];                          //一维动态规划数组
  memset(dp,0,sizeof(dp));                //置边界情况
  for (int i=1;i<=n;i++)
  {  for (int r=W;r>=w[i-1];r--)           //r按0到W的逆序（重点）
      dp[r]=max(dp[r],dp[r-w[i-1]]+v[i-1]);
  }
  return dp[W];
}
```

2 完全背包问题

- 有 n 种重量和价值分别为 w_i 、 v_i ($0 \leq i < n$) 的物品，从这些物品中挑选总重量不超过 W 的物品，每种物品可以挑选任意多件。
- 求挑选物品的最大价值。

解

- 设置动态规划二维数组 dp ， $dp[i][r]$ 表示从物品 $0 \sim i-1$ （共 i 个物品）中选出重量不超过 r 的物品的最大总价值。
- 显然有 $dp[i][0]=0$ （背包不能装入任何物品时，总价值为0）， $dp[0][j]=0$ （没有任何物品可装入时，总价值为0），将它们作为边界情况，为此采用`memset`函数一次性将 dp 数组初始化为0。
- 另外设置二维数组 fk ，其中 $fk[i][r]$ 存放 $dp[i][r]$ 得到最大值时物品 $i-1$ 挑选的件数。考虑物品 $i-1$ ，可以选择0到 k （ $k \times w[i-1] \leq r$ ）次。



状态转移方程

$$dp[i][r] = \max_{k \times w[i-1] \leq r} \{ dp[i-1][r - k \times w[i-1]] + k \times v[i-1] \}$$

$$fk[i][r] = k;$$

物品 $i-1$ 取 k 件

$$n=3, W=7, w=(3, 4, 2), v=(4, 5, 3)$$

$\begin{matrix} j \\ i \end{matrix}$	0	1	2	3	4	5	6	7
0	0[0]	0[0]	0[0]	0[0]	0[0]	0[0]	0[0]	0[0]
1	0[0]	0[0]	0[0]	4[1]	4[1]	4[1]	8[2]	8[2]
2	0[0]	0[0]	0[0]	4[0]	5[1]	5[1]	8[0]	9[1]
3	0[0]	0[0]	3[1]	4[0]	6[2]	7[1]	9[3]	10[2]

最优方案：物品2挑选2件，物品1挑选0件，物品0挑选1件。

```

int n=3,W=7;
int w[MAXN]={3,4,2};
int v[MAXN]={4,5,3};
int dp[MAXN][MAXW],fk[MAXN][MAXW];           //二维动态规划数组

int completeKnap()                           //求解完全背包问题
{
    memset(dp,0,sizeof(dp));
    memset(fk,0,sizeof(fk));
    for (int i=1;i<=n;i++)
    {
        for (int r=0;r<=W;r++)
        {
            for (int k=0;k*w[i-1]<=r;k++)
            {
                if (dp[i][r]<dp[i-1][r-k*w[i-1]]+k*v[i-1])
                {
                    dp[i][r]=dp[i-1][r-k*w[i-1]]+k*v[i-1];
                    fk[i][r]=k;                //物品i-1取k件
                }
            }
        }
    }
    return dp[n][W];
}

```

```
void getx()
```

```
{  int i=n,r=W;
```

```
    while (i>=1)
```

```
    {  printf("物品 %d共%d件 ",i-1,fk[i][r]);
```

```
        r-=fk[i][r]*w[i-1];
```

```
        i--;
```

```
    }
```

```
    printf("\n");
```

```
}
```

//推导一个最优方案

//剩余重量

【算法分析】 completeKnap算法中包含三重循环， k 的循环最坏可能从0到 W ，所以算法的时间复杂度为 $O(nW^2)$ 。

- 将完全背包问题转换为这样的0/1背包问题，物品 i 出现 $\lfloor W/w[i] \rfloor$ 次。
- 例如对于完全背包问题 $n=3$, $W=7$, $w=(3, 4, 2)$, $v=(4, 5, 3)$, 物品0最多取 $W/3=2$ 次, 物品1最多取 $W/4=1$ 次, 物品2最多取 $W/2=3$ 次, 对应的0/1背包问题是 $W=7$, $n=6$, $w=(3, 3, 4, 2, 2, 2)$, $v=(4, 4, 5, 3, 3, 3)$ 。
- 后者的最大价值与前者是相同的。

```

int completeKnap1()                                //改进算法
{
    memset(dp,0,sizeof(dp));
    for (int i=1;i<=n;i++)
    {
        for (int r=0;r<=W;r++)
        {
            if (r<w[i-1])                          //物品i-1放不下
                dp[i][r]=dp[i-1][r];
            else                                     //在不选择和选择物品i-1（多次）中求最大值
                dp[i][r]=max(dp[i-1][r],dp[i][r-w[i-1]]+v[i-1]);
        }
    }
    return dp[n][W];                                //返回总价值
}

```

【算法分析】 上述算法中包含两重循环，所以算法的时间复杂度为 $O(nW)$ 。

算法空间优化

```
int completeKnap2()
{
    int dp[MAXN];
    memset(dp,0,sizeof(dp));
    for (int i=1;i<=n;i++)
    {
        for (int r=w[i-1];r<=W;r++)
            dp[r]=max(dp[r],dp[r-w[i-1]]+v[i-1]);
    }
    return dp[W];
}
```

//完全背包问题改进算法
//一维动态规划数组
//置边界情况

//r按w[i-1]到W的顺序

```
int Knap1()
{
    int dp[MAXN];
    memset(dp,0,sizeof(dp));
    for (int i=1;i<=n;i++)
    {
        for (int r=W;r>=0;r--)
        {
            if (r<w[i-1])
                dp[r]=dp[r];
            else
                dp[r]=max(dp[r],dp[r-w[i-1]]+v[i-1]);
        }
    }
    return dp[W];
}
```

//0/1背包问题改进算法
//一维动态规划数组
//置边界情况

//r按0到W的逆序（重点）

差别

3 多重背包问题*

- 有 n 种重量和价值分别为 w_i 、 v_i ($0 \leq i < n$) 的物品，物品 i 有 s_i 件。从这些物品中挑选总重量不超过 W 的物品，每种物品可以挑选多件，求挑选物品的最大价值。
- 例如， $n=3$ ， $W=7$ ， $w=\{3, 4, 2\}$ ， $v=\{4, 5, 3\}$ ， $s=\{2, 2, 1\}$ ，对应的最大价值为9，一个最优方案是选择物品0和1各一件。

解

- 设置动态规划二维数组 dp ， $dp[i][r]$ 表示从物品 $0 \sim i-1$ （共 i 个物品）中选出重量不超过 r 的物品的最大总价值，选择物品 i 的最多件数为 $s[i]$ 。

```
int multiKnap()
```

```
{  int dp[MAXN][MAXW];
```

```
  memset(dp,0,sizeof(dp));
```

```
  for (int i=1;i<=n;i++)
```

```
  {  for (int r=0;r<=W;r++)
```

```
    {  for (int k=0;k<=s[i-1];k++)
```

```
      {  if(k*w[i-1]<=r)
```

```
        dp[i][r]=max(dp[i][r],dp[i-1][r-k*w[i-1]]+k*v[i-1]);
```

```
      }
```

```
    }
```

```
  }
```

```
  return dp[n][W];
```

```
}
```

//求解多重背包问题

//二维动态规划数组

//不超重时

背包问题总结



三省吾身